

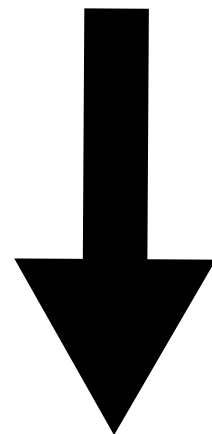
Cache的引入

CPU 的运算速度远高于主存访问速度。当指令需要的数据不在 CPU 附近时，CPU 必须等待内存返回数据。

程序通常不会随机访问整个内存空间，而是集中访问一小部分数据：

CPU: 计算 | 等待数据 | 计算
Memory: |---访问---|

时间局部性：最近访问的数据可能再次访问
空间局部性：相邻的数据可能随后被访问



在 CPU 与内存之间加入
Cache

cache: 小、快，保存近期使用的数据
Memory: 大、慢，保存完整数据

Hit (命中)：数据在 Cache 中，快速返回

Miss (缺失)：数据不在 Cache 中，从内存取回并放入 Cache

Cache 利用程序局部性，将大部分访存转换为高速访问，从而缓解 CPU 与内存之间的速度差异

Cache的映射

| Tag | Index | Offset |

Cache 容量远小于内存，因此需要根据地址确定数据在 Cache 中的位置

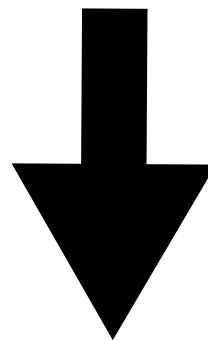
Offset: 选择 Cache Line (缓存行) 内的数据

Index: 选择 Cache 中的 Set (组)

Tag: 判断该位置是否保存目标地址

直接映射: 每个内存地址只能映射到一个固定的 Cache 位置

查找简单、但容易冲突



组相连映射: 为每个地址提供多个候选位置

Set 0: Way 0 | Way 1 | Way 2 | Way 3

Set 1: Way 0 | Way 1 | Way 2 | Way 3

Cache的替换

当一个 Set 的所有 Way 都被占用时，需要 Replacement Policy（替换策略）选择牺牲者

Random：随机替换

LRU：替换最久未使用的 Way

PLRU：用较低成本近似 LRU

Cache Miss

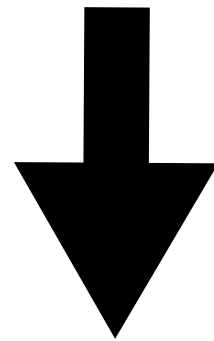
在 Blocking Cache（阻塞式缓存）中，发生 Miss 后，Cache 必须等待下一级返回数据

请求A: Miss — 等待数据 — 完成

请求B: 被阻塞

请求C: 被阻塞

一个请求正在等待内存，不代表整个 Cache 都无法处理其他请求



将等待状态从处理流水线中分离

MSHR（Miss Status Holding Register，缺失状态暂存表）用于记录尚未完成的 Miss

MSHR 缺失状态暂存表

一项 MSHR 通常记录：

Miss地址

请求类型

请求来源

当前处理状态

数据返回后的响应目标

基本流程：

请求发生Miss



分配一个MSHR



向下一级发送请求



流水线继续处理其他请求



数据返回并找到对应MSHR



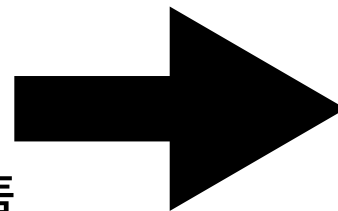
完成回填和响应

MSHR 让 Cache 不必一直占用主流流水线等待数据

MSHR 将 Miss 从一次阻塞式等待，转变成可跟踪、可并发的未完成事务

当 Cache 中存在多个 MSHR 时，不同地址的 Miss 可以并行处理

对于同一个 Cache Line 的后续请求，也可以合并到已有 MSHR



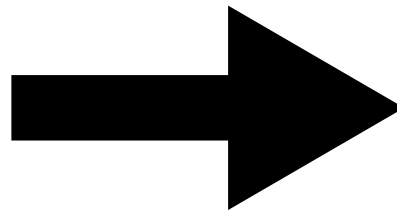
从单级 Cache 到多级 Cache

小容量Cache:

- 距离 CPU 近
- 查询速度快
- 容量有限, Miss 较多

大容量Cache:

- 可以保存更多数据
- 命中率通常更高
- 查询和数据访问延迟更长



用多个层级分别承担速度与容量需求

CPU



L1 Cache: 最小、最快



L2 Cache: 更大、较慢



LLC: 容量更大、通常由多个核心共享



Memory: 最大、最慢

越靠近 CPU, 容量越小、速度越快; 越靠近内存, 容量越大、速度越慢

多级 Cache 用层次化设计同时获得接近小 Cache 的速度和接近大 Cache 的命中能力

Bank、Slice、仲裁与流水化

Cache容量和请求数量增长



单体结构出现端口、时序和吞吐量瓶颈



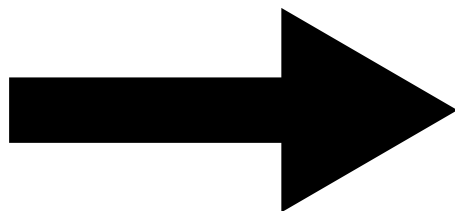
Bank、Slice、仲裁与流水化

现代 Cache 不仅处理 CPU 的读写，还要处理：

普通读写
Miss回填
脏行回写
预取请求
一致性请求

如果所有请求都访问同一个 SRAM 和处理逻辑：

- SRAM 端口成为瓶颈
- 多个请求无法同时访问
- 大容量结构带来更长的连线和查询延迟
- 复杂处理路径限制工作频率



用 Bank 和 Slice 分散压力

仲裁与流水化提高吞吐量

Bank（存储体）

将 Data Array（数据阵列）拆成
多个相对独立的 SRAM：

请求A → Bank 0

请求B → Bank 1

请求C → Bank 2

地址映射与分发：访问请求进入缓存后，会通过
一个特定的算法来决定它应该去往哪个bank

独立并行访问：每个bank都是一个独立的功能单元，拥有
自己的读/写端口和存储队列，可以同时响应不同的请求

Slice（切片）

将 Cache 按地址划分成多个处理单元：

Cache

- Slice 0: Directory + Data + Pipeline
- Slice 1: Directory + Data + Pipeline
- Slice 2: Directory + Data + Pipeline
- Slice 3: Directory + Data + Pipeline

每个 Slice 独立负责一部分地址，分散请求和状态管理压力

仲裁与流水化提高吞吐量

当多个请求访问同一资源时，由 Arbiter
(仲裁器) 选择当前允许进入的请求：

CPU请求

回填请求 → Arbiter → Cache资源

回写请求

预取请求

- Bank 增加存储并行度
- Slice 分散地址和处理压力
- 仲裁解决资源竞争
- 流水线提高频率与吞吐量

流水线则将复杂操作拆分为多个阶段：

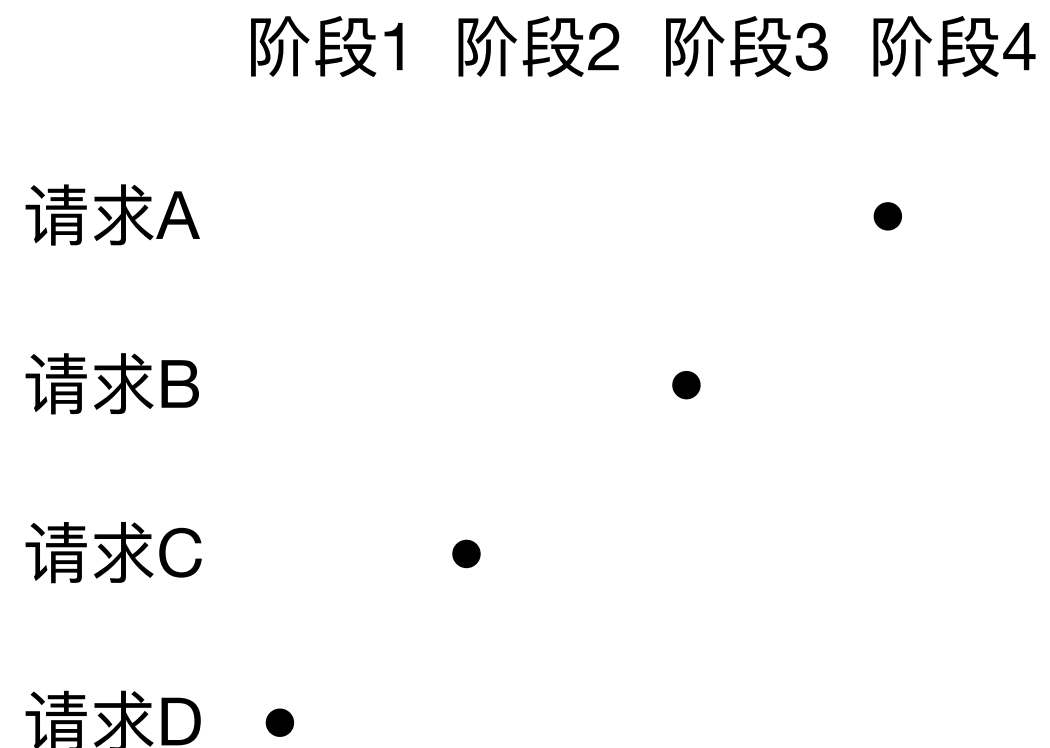
周期1：请求选择

周期2：Directory查询

周期3：命中判断与状态处理

周期4：Data访问与响应

不同请求可以同时处于不同阶段：

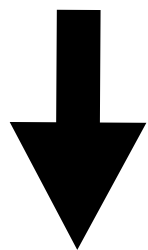


主动预取

非阻塞 Cache 可以同时处理多个 Miss

Miss A ————— 等待数据
Miss B ————— 等待数据
Miss C ————— 等待数据

这提高了并行度，但当 CPU 首次访问某个缺失数据时，仍然需要等待



从发生 Miss 后取数，
变为使用数据前取数

主动预取：

观察访问规律->
预测未来地址->
提前访问下一级->
数据进入Cache->
CPU访问时直接命中

有效预取需要满足：

- Accuracy（准确率）：预取的数据最终被使用
- Coverage（覆盖率）：能够覆盖足够多的原始 Miss
- Timeliness（及时性）：数据在 CPU 使用前返回

错误或过度预取会产生：

无用数据进入Cache → 污染Cache
占用下级请求带宽 → 阻塞正常请求
占用MSHR和队列 → 增加资源竞争
驱逐有效数据 → 产生新的Miss

预取通过提前发起访存隐藏 Miss 延迟，但必须在性能收益和资源消耗之间取得平衡

缓存一致性与协议

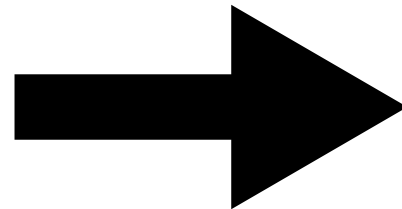
多个 Cache 可能保存同一地址的副本

初始状态:

Core 0 Cache: $X = 0$

Core 1 Cache: $X = 0$

Memory : $X = 0$



Core 0 写入 $X = 1$:

Core 0 Cache: $X = 1$

Core 1 Cache: $X = 0$ ← 旧副本

Memory : $X = 0$

系统必须回答:

- 谁持有最新数据?
- 写入前是否需要让其他副本失效?
- 后续读取应该从哪里获得数据?

MESI算法

MESI

状态	含义
Modified	本地独占，数据已被修改
Exclusive	本地独占，数据仍然干净
Shared	多个 Cache 可以共同读取
Invalid	本地副本无效

时间步	操作	Core 0	Core 1	主存	动作
0	初始	I	I	7	
1	core0读A(未命中)	E (7)	I	7	core 0发读请求 从主存加载 标记E
2	core1读A(未命中)	S (7)	S (7)	7	core1发读请求 core0监听到，E->S core1从主存加载，标记S
3	core0写A=10	M (10)	I	7 (作废)	core0发RFO，core1监听到，S->I core0获独占权,写入10,I->M
4	core1读A	S (10)	S (10)	10	core1发读请求->core0监听到M，执行写回(10->主存)并转发给core1->core0 M->S, core1 I->S
5	core1写A=20	I	M (20)	10	core1发RFO->core0监听到，S->I cor1获独占权，写入20，S->M

从广播 Snoop 到 Directory

写入前，需要找到其他数据副本

一种方法是将请求广播给所有 Cache：

Core 0请求写X

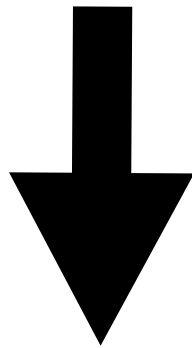


询问所有其他Cache



持有X的Cache将副本失效

核心数量增加后，广播会产生大量无效流量。



Directory（目录）记录每条 Cache Line 的持有情况：

Directory[X]

—— 当前一致性状态

—— 哪些Cache持有副本

—— 哪个Cache可能持有最新数据

Home Node 查询 Directory 后，只向必要的 Cache 发送 Snoop（探测）

CHI

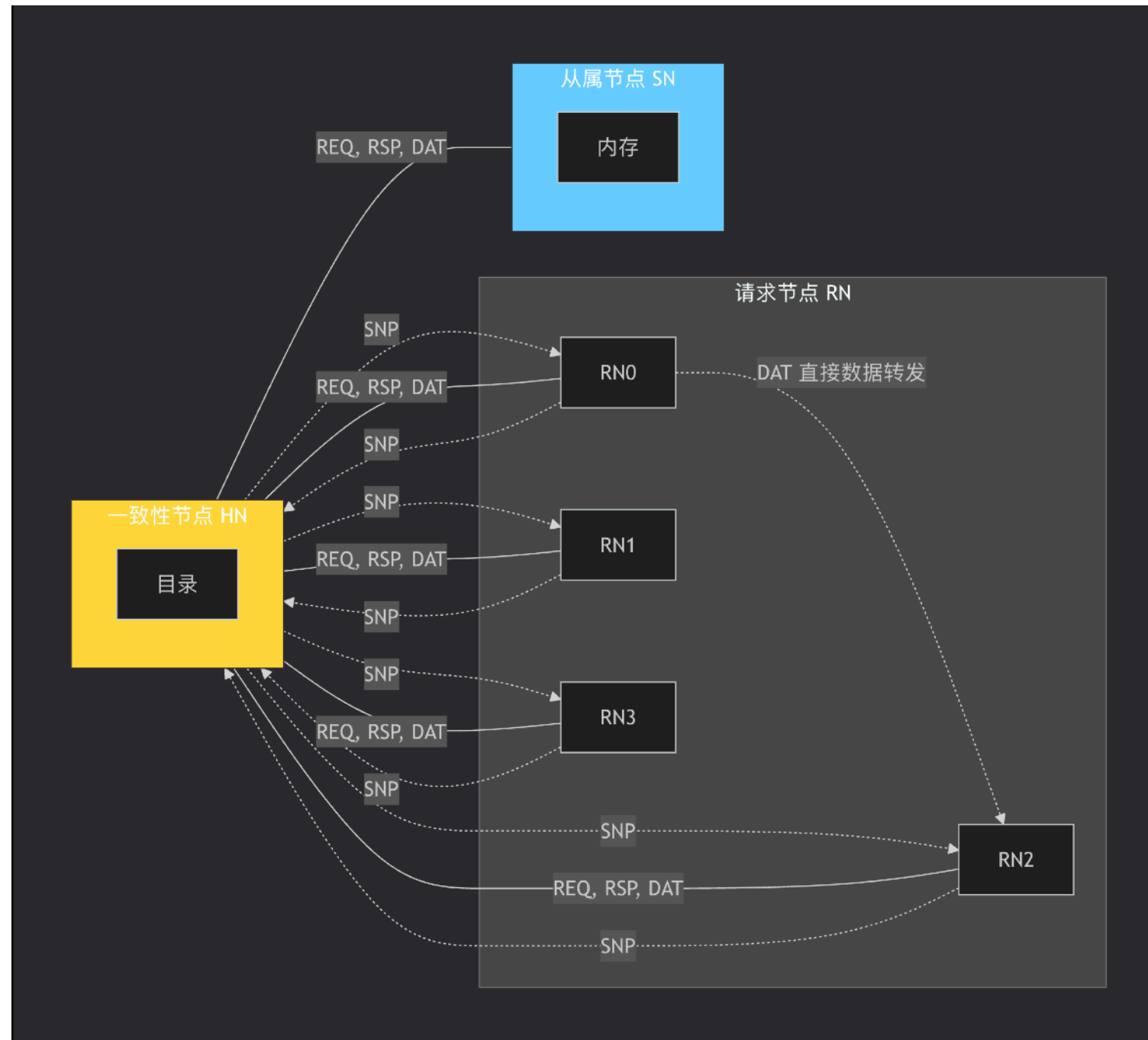
用“目录”和“精准通知”取代了早期协议的“无脑广播”，将通信复杂度从 $O(N^2)$ 降为 $O(N)$

CHI 节点	XSCache 模块	职责
RN-F	CoupledL2	发起一致性请求并缓存数据
HN-F	OpenLLC	管理共享目录和一致性事务
SN-F/Bridge	OpenNCB	连接下游内存

通道	作用
REQ	发出读、写和回写请求
SNP	探测其他 Cache
RSP	返回权限、完成和确认
DAT	传输 Cache Line 数据

一次一致性操作通常不是单个请求，而是多个通道协作完成的事务

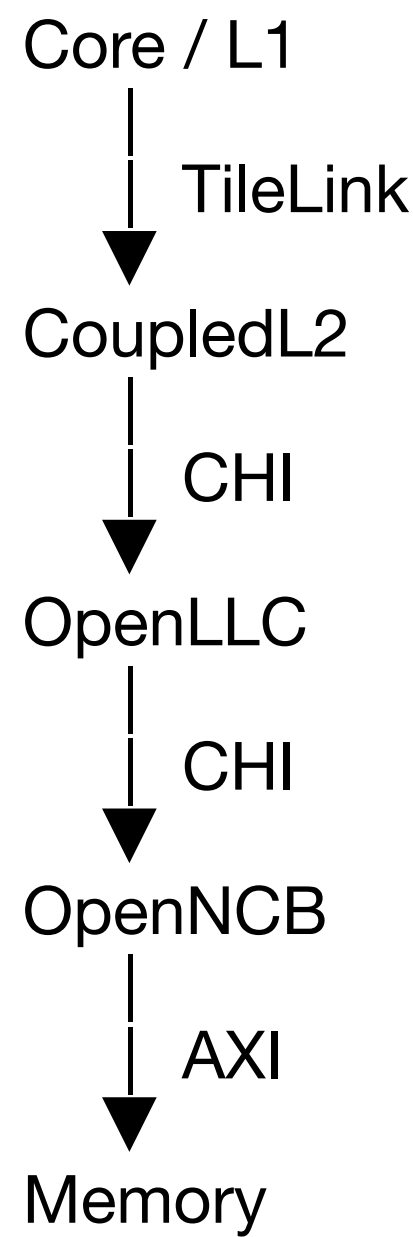
CHI



<https://chat.deepseek.com/share/segjgtnr96qh0h54o8>

不同协议负责不同边界

一致性需要跨越多个系统层级



概念或协议	作用
MESI 类状态模型	描述 Cache Line 的状态与权限
TileLink	连接 L1 与 CoupledL2，传递请求、权限和数据
CHI	连接一致性节点，传递请求、探测、响应和数据
AXI	连接内存系统，完成最终的数据读写

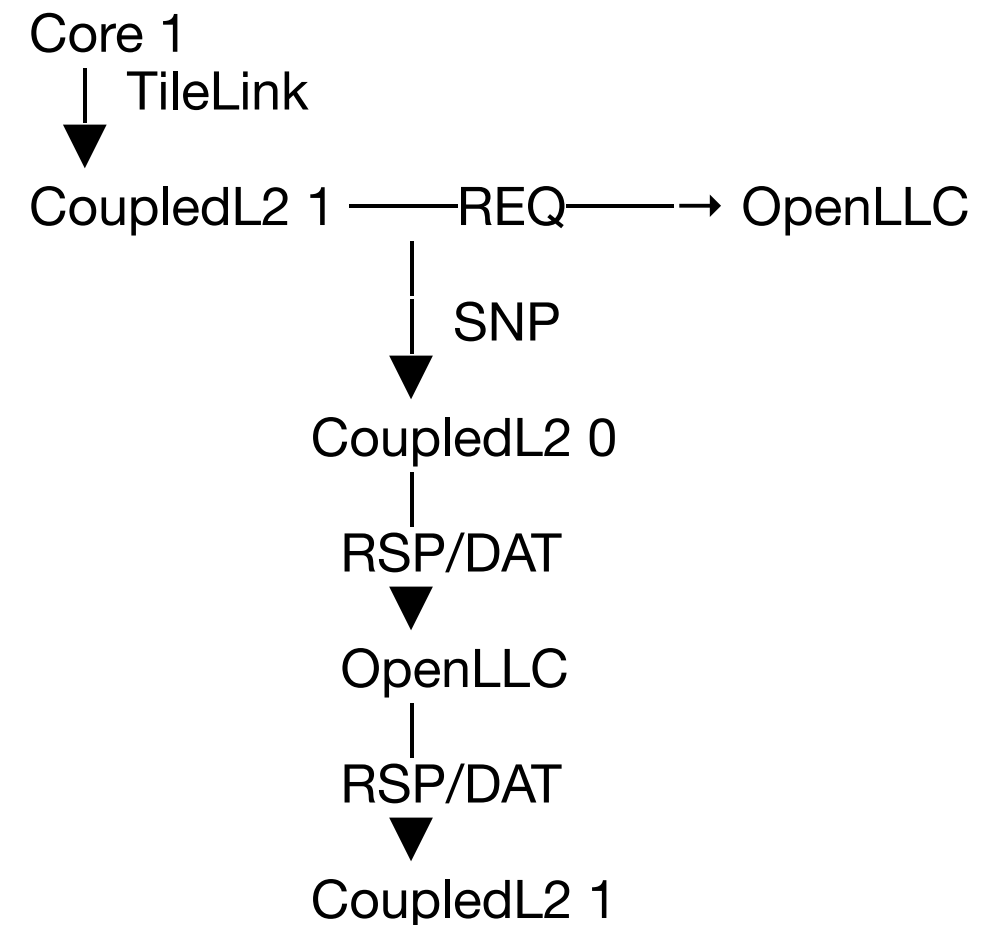
MESI 解释“状态如何变化”，TileLink 和 CHI 负责“如何通过事务完成变化”，AXI 负责最终内存访问

一次写请求串起协议

假设 Core 0 和 Core 1 都持有 X 的只读副本

1. Core 1通过TileLink向CoupledL2请求写X
2. CoupledL2发现本地没有写权限
3. CoupledL2通过CHI REQ向OpenLLC请求Unique权限
4. OpenLLC查询Directory, 发现Core 0持有副本
5. OpenLLC通过CHI SNP要求Core 0失效X
6. Core 0通过CHI RSP/DAT返回响应或最新数据
7. OpenLLC更新Directory
8. OpenLLC向Core 1返回数据并授予写权限
9. CoupledL2通过TileLink完成Core 1的请求

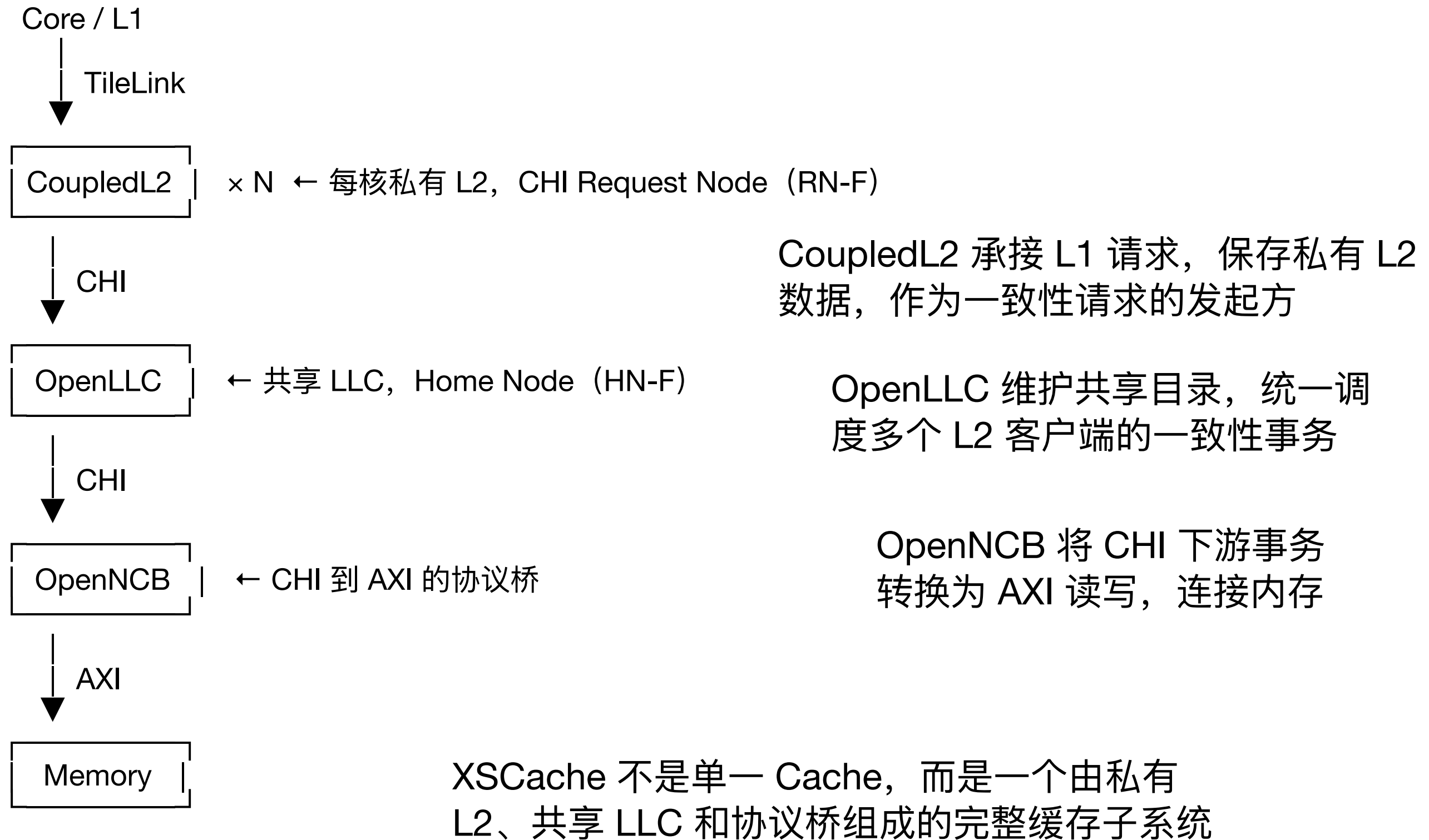
事务路径:



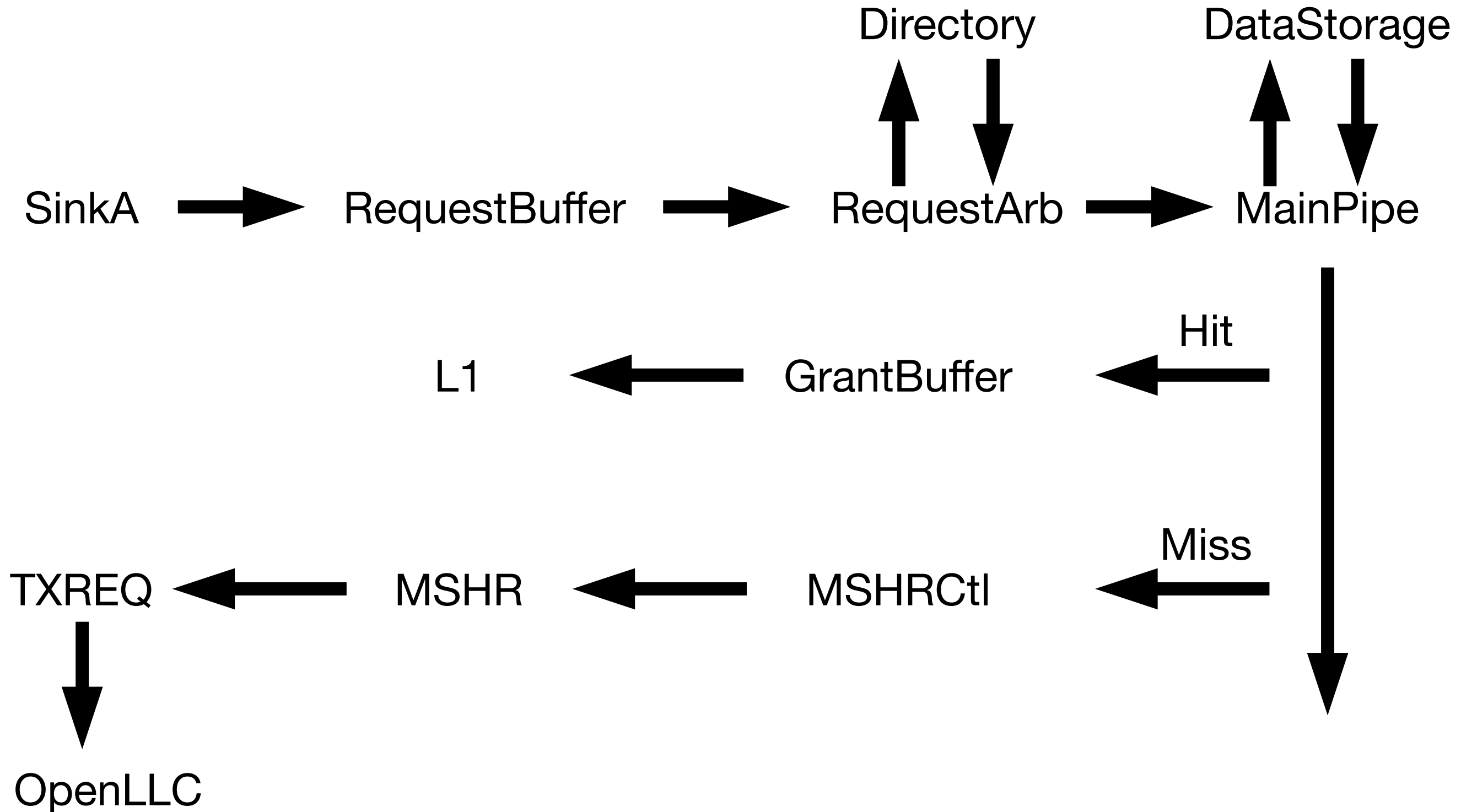
缓存一致性不是某一个模块单独完成的, 而是状态机、Directory、Snoop 和多个协议共同完成的分布式事务

XSCache

CHI-only Cache 子系统，连接 TileLink 与 AXI 内存



CoupledL2数据流



SinkA

接收 L1 发来的 TileLink A 通道请求，转换为内部统一格式 TaskBundle

```
// SinkA.scala:30-35 — IO 定义
class SinkA(implicit p: Parameters) extends L2Module {
  val io = IO(new Bundle() {
    val a = Flipped(DecoupledIO(new TLBundleA(edgeIn.bundle))) // ← TL A 输入
    val task = DecoupledIO(new TaskBundle) // → TaskBundle 输出
  })
```

```
// SinkA.scala:54-68 — TL → TaskBundle 转换
def fromTLAtoTaskBundle(a: TLBundleA): TaskBundle = {
  val task = Wire(new TaskBundle)
  task.channel := "b001".U // 标记来源: A 通道
  task.tag := parseAddress(a.address)._1 // 地址 → Tag
  task.set := parseAddress(a.address)._2 // 地址 → Set
  task.off := parseAddress(a.address)._3 // 地址 → Offset
  task.opcode := a.opcode // 操作类型: Get/AcquireBlock/...
  task.param := a.param
  task.size := a.size
  task.sourceId := a.source // L1 请求 ID, 用于返回时匹配
  task.corrupt := a.corrupt
  task.mshrTask := false.B // 新请求, 不是 MSHR 重试
  task.wayMask := 0.U(cacheParams.ways.W)
  task.reqSource := a.user.lift(utility.ReqSourceKey)
    .getOrElse(MemReqSource.NoWhere.id.U)
  task
}
```

RequestBuffer

暂存来自 SinkA 的 TaskBundle。如果一个地址已经有 MSHR 在等待（Miss 状态），后续相同地址的请求可以合并到同一个 MSHR，避免重复发起 CHI 请求

```
// RequestBuffer.scala:30-41 — 缓存条目结构
class ReqEntry(entries: Int = 4)(implicit p: Parameters) extends L2Bundle() {
  val valid    = Bool()
  val rdy      = Bool()
  val task     = new TaskBundle()

  /* blocked by MainPipe:
   * [3] by stage1, a same-set entry just fired
   * [2] by stage2
   * [1] by stage3
   * [0] block release flag
   */
  val waitMP   = UInt(4.W)
}
```

RequestArb

从四个入口（SinkA、SinkB、SinkC、MSHR）中仲裁选出一个请求。同一拍并发发起 Directory 查询

```
// RequestArb.scala:29-49 — IO 定义
class RequestArb(implicit p: Parameters) extends L2Module {
  val io = IO(new Bundle() {
    /* 四个输入来源 */
    val sinkA      = Flipped(DecoupledIO(new TaskBundle)) // L1 请求
    val sinkB      = Flipped(DecoupledIO(new TaskBundle)) // Snoop 探测
    val sinkC      = Flipped(DecoupledIO(new TaskBundle)) // 释放/写回
    val mshrTask   = Flipped(DecoupledIO(new TaskBundle)) // MSHR 重试

    /* 输出 */
    val dirRead_s1 = DecoupledIO(new DirRead())           // → Directory 查询
    val taskToPipe_s2 = ValidIO(new TaskBundle())         // → MainPipe
  })
}
```


Directory

保存 Cache Line 的 Tag 和状态

```
// Directory.scala:31-49 — 每行的元数据
class MetaEntry(implicit p: Parameters) extends L2Bundle {
  val dirty    = Bool()           // 是否脏数据
  val state    = UInt(stateBits.W) // 一致性状态
  val clients  = UInt(clientBits.W) // 哪些 L1 Client 持有
  val prefetch = if (hasPrefetchBit) Some(Bool()) else None
  val accessed = Bool()           // 用于替换策略
  val tagErr   = Bool()           // Tag ECC 错误
  val dataErr  = Bool()
}

// Directory.scala:75-80 — 查询请求
class DirRead(implicit p: Parameters) extends L2Bundle {
  val tag      = UInt(tagBits.W)
  val set      = UInt(setBits.W)
  val wayMask  = UInt(cacheParams.ways.W) // 允许匹配的 Way 范围
  val replacerInfo = new ReplacerInfo()
}

// Directory.scala:89-97 — 查询结果
class DirResult(implicit p: Parameters) extends L2Bundle {
  val hit      = Bool()           // 是否命中
  val tag      = UInt(tagBits.W)
  val set      = UInt(setBits.W)
  val way      = UInt(wayBits.W)   // 命中的 Way 号 (或替换候选 Way)
  val meta     = new MetaEntry()   // 命中时的元数据
  val error    = Bool()
}
```


DataStorage

保存 Cache Line 的实际数据 (64 字节)

```
// DataStorage.scala:26-39 — 数据结构
class DSRequest(implicit p: Parameters) extends L2Bundle {
  val way = UInt(wayBits.W)
  val set = UInt(setBits.W)
  val wen = Bool() // write enable, false=读, true=写
}

class DSBlock(implicit p: Parameters) extends L2Bundle {
  val data = UInt(blockBits.W) // 64字节数据块
}

// DataStorage.scala:50-66 — IO 定义
class DataStorage(implicit p: Parameters) extends L2Module {
  val io = IO(new Bundle() {
    val en = Input(Bool()) // 读写使能
    val req = Flipped(ValidIO(new DSRequest)) // 输入: way + set + wen
    val rdata = Output(new DSBlock) // 输出: 读到的数据
    val wdata = Input(new DSBlock) // 输入: 写入的数据
  })
}
```

MainPipe

流水线核心模块。s2 接收请求，s3 根据 Directory 结果判断命中/缺失，s4 发起后续动作，s5 收 DataStorage 数据，s6 打包发送

```
// MainPipe.scala:148 — s2: 接收 RequestArb 选中的请求
val task_s2 = io.taskFromArb_s2

// MainPipe.scala:152-161 — s3: 打入寄存器，收 Directory 结果
val task_s3 = RegInit(0.U.asTypeOf(Valid(new TaskBundle)))
task_s3.valid := task_s2.valid
task_s3.bits := task_s2.bits

val dirResult_s3 = io.dirResp_s3 // Directory 查询结果
val meta_s3      = dirResult_s3.meta

// MainPipe.scala:243-264 — s3: 核心判断：是否需要 MSHR
val acquire_on_miss_s3 = req_acquire_s3 || req_prefetch_s3 || req_get_s3
val acquire_on_hit_s3  = metaOnHit_s3.state === BRANCH && req_needT_s3

val need_acquire_s3_a = req_s3.fromA && Mux(
  dirResult_s3.hit,
  acquire_on_hit_s3, // 命中时判断：要不要升级权限
  acquire_on_miss_s3 // 缺失时：需要 MSHR
)

val need_mshr_s3 = need_acquire_s3_a || need_probe_s3_a || cache_alias
// → true: 走 Miss 路径，分配 MSHR
// → false: 走 Hit 路径，MainPipe 内部完成

// MainPipe.scala:800-837 — s4: 打包输出
txreq_s4.bits := task_s4.bits.toCHIREQBundle() // Miss → CHI REQ
d_s4.bits.task := task_s4.bits                // Hit → GrantBuffer
d_s4.bits.data.data := data_s4                 // 附带数据

// MainPipe.scala:840-863 — s5: 收 DataStorage 读结果
val rdata_s5 = io.toDS.rdata_s5.data // 64 字节
```

GrantBuffer (Hit 路径终点)

接收 MainPipe 发来的 TaskBundle + 数据, 打包成 TileLink D 通道格式发回 L1。同时等待 L1 的 TL E 确认 (GrantAck)

```
// GrantBuffer.scala:59-66 — IO 定义
class GrantBuffer(implicit p: Parameters) extends L2Module {
  val io = IO(new Bundle() {
    val d_task = Flipped(DecoupledIO(new TaskWithData())) // ← MainPipe
    val d = DecoupledIO(new TLBundleD(edgeIn.bundle)) // → L1 (TL D)
    val e = Flipped(DecoupledIO(new TLBundleE(edgeIn.bundle))) // ← L1 (TL E 确认)
  })

// GrantBuffer.scala:86-98 — TaskBundle → TL D 打包
  def toTLBundleD(task: TaskBundle, data: UInt = 0.U,
    grant_id: UInt = 0.U) = {
    val d = Wire(new TLBundleD(edgeIn.bundle))
    d.opcode := task.opcode // GrantData
    d.param := task.param
    d.source := task.sourceId // 对应 L1 原始请求 ID
    d.sink := grant_id
    d.denied := task.denied
    d.data := data // 64 字节
    d.corrupt := task.corrupt || task.denied
    d
  }
}
```

MSHRCtl + MSHR (Miss 路径)

MSHRCtl 作用：管理所有 MSHR 条目的分配。
有闲置条目才允许 MainPipe 分配新请求

MSHR 作用：维护一个未完成请求的状态机。
核心一步：将 TL 操作码映射为 CHI 操作码

```
// MSHR.scala:374-392 — TL → CHI 操作码映射
/**
 *   TL                               →   CHI
 *   -----
 *   *   Get                           →   ReadNotSharedDirty
 *   *   AcquireBlock NtoB              →   ReadNotSharedDirty
 *   *   AcquireBlock NtoT              →   ReadUnique
 *   *   AcquirePerm NtoT               →   MakeUnique
 *   *   AcquirePerm BtoT               →   MakeUnique
 *   *   PrefetchRead                   →   ReadNotSharedDirty
 *   *   PrefetchWrite                  →   ReadUnique
 */
oa.opcode := ParallelPriorityMux(Seq(
  release_valid2          -> req_released_chiOpcode,
  req_cboClean             -> CleanShared,
  req_cboFlush             -> CleanInvalid,
  req_cboInval             -> MakeInvalid,
  req_acquirePerm          -> MakeUnique,
  req_needT                -> ReadUnique,
  req_needB /* Default */ -> ReadNotSharedDirty
))

oa.size    := log2Ceil(blockBytes).U
oa.addr    := Cat(req.tag, req.set, 0.U(offsetBits.W))
oa.order   := OrderEncodings.None
```

TXREQ (Miss 路径终点)

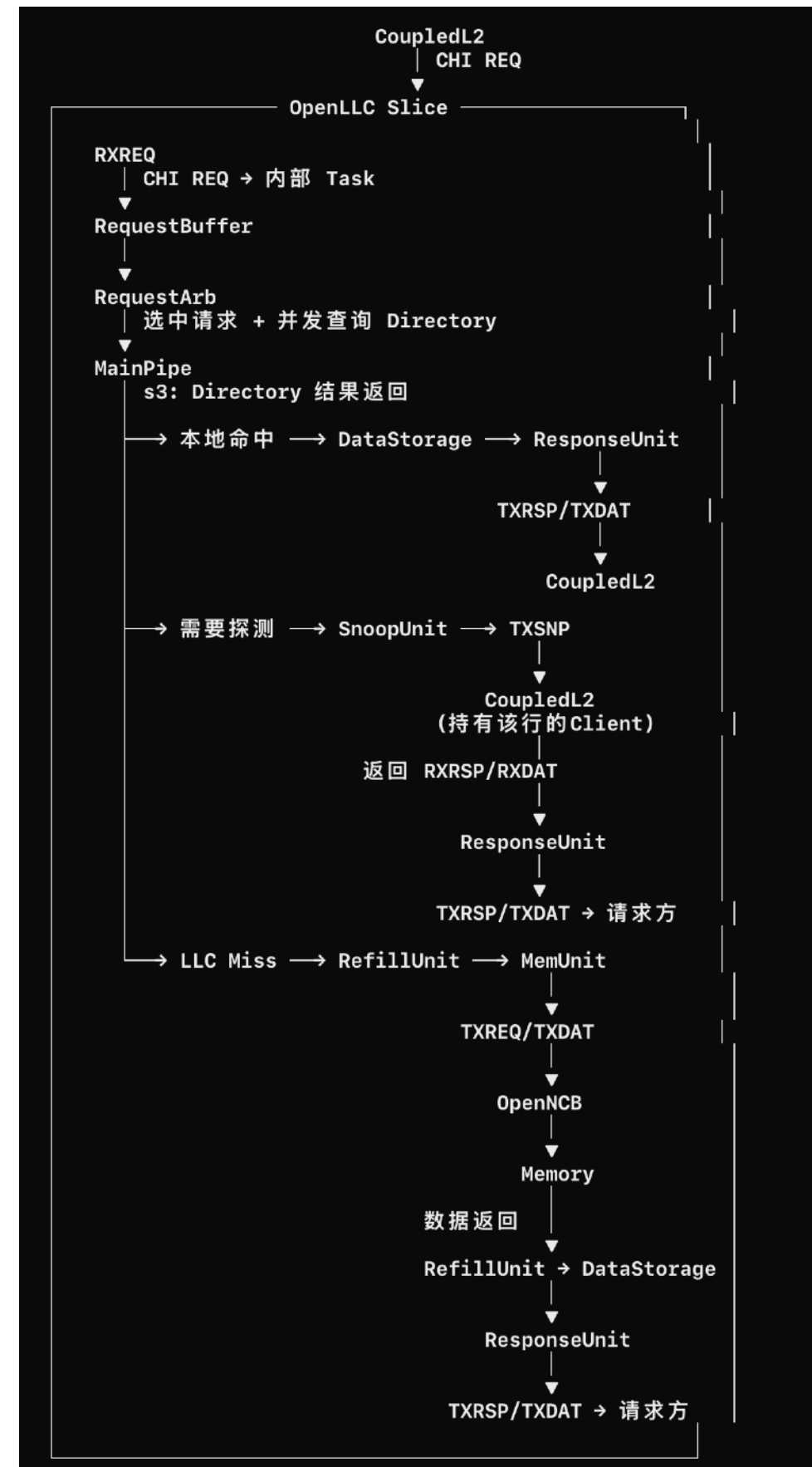
接收 MainPipe 和 MSHR 两个来源的 CHI REQ，内部 Queue 缓冲后发往下游 OpenLLC

```
// TXREQ.scala:33-43 — IO 定义
class TXREQ(implicit p: Parameters) extends CoupledL2Module {
  val io = IO(new Bundle() {
    val pipeReq = Flipped(DecoupledIO(new CHIREQ())) // ← MainPipe
    val mshrReq = Flipped(DecoupledIO(new CHIREQ())) // ← MSHR
    val out      = DecoupledIO(new CHIREQ())          // → OpenLLC
  })

// TXREQ.scala:49,72-77 — Queue 缓冲并合并
  val queue = Module(new Queue(new CHIREQ,
                                entries = mshrsAll, flow = false))

  queue.io.enq.valid := io.pipeReq.valid || io.mshrReq.valid && !noSpace
  queue.io.enq.bits  := Mux(io.pipeReq.valid,
                            io.pipeReq.bits, io.mshrReq.bits)
  io.out <> queue.io.deq
```

OpenLLC数据流



OpenLLC 顶层：入口分发

：接收多个 CoupledL2 Client 的 CHI 请求，通过 RNLinkMonitor 和 RNXbar 分发到对应 Slice。同时通过 SNLinkMonitor 连接下游 OpenNCB/N内存

```
// OpenLLC.scala:37-46 — IO 定义
val io = IO(new Bundle() {
  val rn = Vec(numRNs, Flipped(new PortIO)) // ← 接收 N 个 CoupledL2 的 CHI
  val sn = new NoSnpPortIO                 // → 下游 OpenNCB / 内存
  val nodeID = Input(UInt())
})

// OpenLLC.scala:57-67 — 内部组织
val mmioDiverger = Module(new MMIODiverger()) // 分离可缓存/不可缓存请求
val mmioMerger   = Module(new MMIOMerger())   // 合并可缓存/不可缓存响应
val rnXbar       = Module(new RNXbar())       // 交叉开关 → 分发到各 Slice
val snXbar       = Module(new SNXbar())

val slices = (0 until banks).map { i =>
  val slice = Module(new Slice())
  slice.io.in <> rnXbar.io.out(i) // 每个 Slice 接入
  snXbar.io.in(i) <> slice.io.out // Slice 下游出自 SNXbar
  slice
}
```


RXREQ: CHI → 内部 Task

接收 CHI REQ, 转换为 OpenLLC 内部统一格式 Task。是所有模块传递的统一数据包

```
// RXREQ.scala:26-30 — IO 定义
class RXREQ (implicit p: Parameters) extends LLCModule {
  val io = IO(new Bundle() {
    val req  = Flipped(DecoupledIO(new CHIREQ())) // ← CHI REQ 输入
    val task = DecoupledIO(new Task())           // → Task 输出
  })

// RXREQ.scala:53-77 — CHI REQ → Task 转换
def fromCHIREQtoTaskBundle(r: CHIREQ): Task = {
  val task = Wire(new Task)
  val (tag, set, bank, off) = parseAddress(r.addr)
  task.tag      := tag
  task.set      := set
  task.bank     := bank
  task.off      := off
  task.size     := r.size
  task.chiOpcode := r.opcode // CHI 操作码
  task.txnID    := r.txnID   // 事务 ID
  task.srcID    := r.srcID   // 请求方 ID
  task.tgtID    := r.tgtID   // 目标方 ID
  task.allowRetry := r.allowRetry
  task.memAttr   := r.memAttr
  task
}
```


Task: OpenLLC 内部统一数据结构

```
// Common.scala:37-73 — Task 定义
class Task(implicit p: Parameters) extends LLCBundle {
  val set = UInt(setBits.W)
  val bank = UInt(bankBits.W)
  val tag = UInt(tagBits.W)
  val off = UInt(offsetBits.W)
  val size = UInt(SIZE_WIDTH.W)

  // 身份信息
  val reqID = UInt(TXNID_WIDTH.W) // LLC 内部事务 ID
  val txnID = UInt(TXNID_WIDTH.W) // CHI 事务 ID
  val srcID = UInt(SRCID_WIDTH.W) // 来源节点 ID
  val refillTask = Bool() // 是否为回填任务

  // Snoop 信息
  val snpVec = Vec(numRNs, Bool()) // 需要探测哪些 Client 的位向量
  val replSnp = Bool() // 是否由替换触发的探测

  // CHI 信息
  val chiOpcode = UInt(OPCODE_WIDTH.W) // 操作码
  val dbID = UInt(DBID_WIDTH.W)
  val resp = UInt(RESP_WIDTH.W)
  val retToSrc = Bool()
  val expCompAck = Bool()
  val allowRetry = Bool()
  val memAttr = new MemAttr
}
```

RequestBuffer

暂存来自 RXREQ 的 Task，内部数组缓冲区，先进先出

```
// RequestBuffer.scala:26-30 — IO 定义
class RequestBuffer(entries: Int = 8)(implicit p: Parameters)
  extends LLCModule with HasCHIOpcodes {
  val io = IO(new Bundle() {
    val in  = Flipped(DecoupledIO(new Task())) // ← RXREQ
    val out = DecoupledIO(new Task())          // → RequestArb
  })

// RequestBuffer.scala:33-48 — 内部数组缓冲
  val buffer = RegInit(VecInit(
    Seq.fill(entries)(0.U.asTypeOf(Valid(new Task()))))
  ))
  val insertIdx = PriorityEncoder(buffer.map(!_.valid))
  when(in.fire) {
    buffer(insertIdx).valid := true.B
    buffer(insertIdx).bits  := in.bits
  }
}
```

RequestArb

仲裁两个来源（外部请求 + RefillUnit
回填任务），并发发起 Directory 查询

```
// RequestArb.scala:26-36 — IO 定义
class RequestArb(implicit p: Parameters) extends LLCModule
  with HasClientInfo with HasCHIOpcodes {
  val io = IO(new Bundle() {
    val busTask_s1      = Flipped(DecoupledIO(new Task())) // ← RequestBuffer
    val refillTask_s1   = Flipped(DecoupledIO(new Task())) // ← RefillUnit
    val dirRead_s1      = DecoupledIO(new DirRead())       // → Directory
    val taskToPipe_s2   = ValidIO(new Task())              // → MainPipe
  })
```

Directory

OpenLLC 的 Directory 保存两层信息——自己的 Cache Line 元数据和 Snoop Filter（每个 Client 是否持有该行）。通过查询结果决定：本地命中、是否需要 Snoop、是否需要访问内存

```
// Directory.scala:38-58 — 自身元数据和 Client 元数据
class SelfMetaEntry(implicit p: Parameters) extends Bundle {
  val valid = Bool()
  val dirty = Bool()
}

class ClientMetaEntry(implicit p: Parameters) extends Bundle {
  val valid = Bool()
  // 记录每个 Client 是否持有该 Cache Line
}
```

DataStorage

保存 LLC 的 Cache Line 数据。支持同周期内独立的读和写

```
// DataStorage.scala:25-53 — 数据结构和 IO
class DSRequest(implicit p: Parameters) extends LLCBundle {
  val way = UInt(wayBits.W)
  val set = UInt(setBits.W)
}

class DSBlock(implicit p: Parameters) extends LLCBundle {
  val data = Vec(beatSize, new DSBeat()) // 多 beat 数据块
}

class DataStorage(implicit p: Parameters) extends LLCModule {
  val io = IO(new Bundle() {
    val read = Flipped(ValidIO(new DSRequest())) // 独立读端口
    val write = Flipped(ValidIO(new DSRequest())) // 独立写端口
    val rdata = Output(new DSBlock()) // 读数据
    val wdata = Input(new DSBlock()) // 写数据
  })
}
```

MainPipe

流水线核心。s2 接收 Task, s3 读 Directory 结果后判断三条路：本地命中、需要 Snoop、LLC Miss 需要访问内存。s4 发起对应模块的请求

```
// MainPipe.scala:27-65 — IO 定义
class MainPipe(implicit p: Parameters) extends LLCModule with HasCHIOpcodes {
  val io = IO(new Bundle() {
    val taskFromArb_s2 = Flipped(ValidIO(new Task())) // s2: ← RequestArb
    val dirResp_s3      = Input(new DirResult())       // s3: ← Directory
    val dirWReq_s3      = new DirWriteIO()             // s3: → Directory 更新

    val snoopTask_s4    = ValidIO(new Task())          // s4: → SnoopUnit
    val refillReq_s4    = ValidIO(new RefillRequest())  // s4: → RefillUnit (Miss)
    val toMemUnit       = ...                          // s4: → MemUnit (内存读写)

    val toResponseUnit = ...                          // s4/s6: → ResponseUnit
    val toDS_s4        = ...                          // s4: → DataStorage
    val rdataFromDS_s6 = Input(new DSBlock())          // s6: ← DataStorage
  })
}
```

SnoopUnit

接收 MainPipe 发来的 Snoop Task，暂存、去重、顺序化后通过 TXSNP 发给持有该 Cache Line 的 L2 Client。相同的地址冲突时会排队等待

```
// SnoopUnit.scala:31-47 — IO 定义
class SnoopUnit(implicit p: Parameters) extends LLCModule with HasCHIOpcodes {
  val io = IO(new Bundle() {
    val in  = Flipped(ValidIO(new Task())) // ← MainPipe
    val out = DecoupledIO(new Task())      // → TXSNP

    // 检查是否与正在处理的响应冲突（同地址已有 ReadUnique 等）
    val respInfo = Flipped(Vec(mshrs.response,
                               ValidIO(new ResponseInfo()))))
    val ack = Flipped(ValidIO(new Resp())) // ← CompAck 解除冲突锁
  })

// SnoopUnit.scala:62 — 冲突检测：同地址已有未完成请求就等待
def snpConflictMask(a: Task): UInt = VecInit(io.respInfo.map(s =>
  s.valid && a.replSnp && sameAddr(a, s.bits)
)).asUInt
```


TXSNP

将 SnoopUnit 发来的内部 Task 转换为 CHI SNP 格式，发给对应 L2 Client

```
// TXSNP.scala:26-37 — IO 定义
class TXSNP (implicit p: Parameters) extends LLCModule {
  val io = IO(new Bundle() {
    val task      = Flipped(DecoupledIO(new Task())) // ← SnoopUnit
    val snp       = DecoupledIO(new CHISNP())         // → CHI SNP 通道
    val snpMask   = Output(Vec(numRNs, Bool()))       // 指示发给哪个 Client
  })

  io.snp.valid := io.task.valid
  io.snp.bits  := io.task.bits.toCHISNPBundle()
  io.snpMask  := io.task.bits.snpVec              // 按 snpVec 发送
  io.task.ready := io.snp.ready
```


RefillUnit

管理 LLC Miss 后的回填过程。分配回填条目，跟踪数据是否返回、是否还需要 Snoop 响应

```
// MemUnit.scala:60 — 模块定义
class MemUnit(implicit p: Parameters) extends LLCModule with HasCHIOpcodes {
  val io = IO(new Bundle() {
    val fromMainPipe = new Bundle() { // ← MainPipe
      val alloc_s4 = Flipped(ValidIO(new MemRequest(withData = false)))
      val alloc_s6 = Flipped(ValidIO(new MemRequest(withData = true)))
    }
    val txreq = DecoupledIO(new CHIREQ()) // → 下游 TXREQ
    val txdat = DecoupledIO(new TaskWithData()) // → 下游 TXDAT
    val urgentRead = Flipped(Vec(beatSize, ValidIO(...))) // ← ResponseUnit 紧急读
    val bypassData = Output(Vec(beatSize, ...)) // → ResponseUnit 旁路数据
  })

// MemUnit.scala:26-32 — 状态机
class MemState(implicit p: Parameters) extends LLCBundle {
  val s_issueReq = Bool() // 发送请求阶段
  val s_issueDat = Bool() // 发送数据阶段
  val w_datRsp = Bool() // 等待数据响应
  val w_dbid = Bool() // 等待 DBID
  val w_comp = Bool() // 等待完成确认
}
```

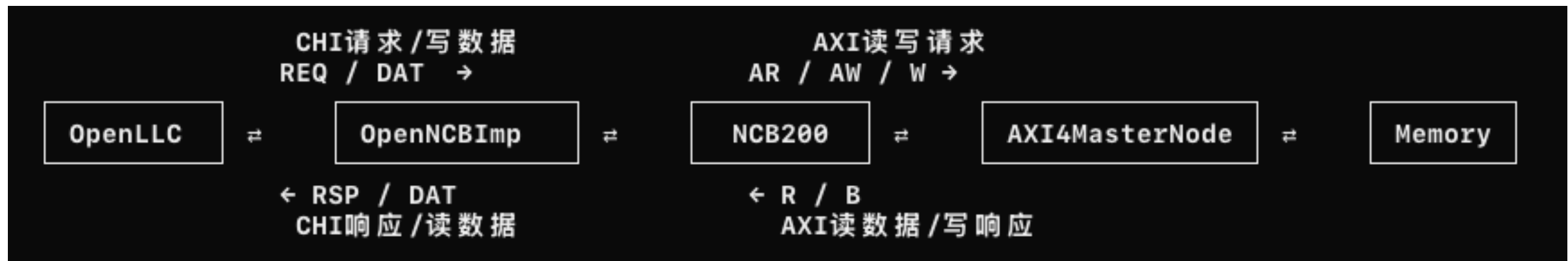
ResponseUnit

统一收集所有响应数据（本地命中数据、RefillUnit 回填数据、MemUnit 旁路数据），按 txnID 匹配后通过 TXRSP/TXDAT 返回原始请求方

```
// ResponseUnit.scala:57-73 — IO 定义
class ResponseUnit(implicit p: Parameters) extends LLCModule with HasCHIOpcodes {
  val io = IO(new Bundle() {
    val fromMainPipe = new Bundle() { // ← MainPipe
      val alloc_s4 = Flipped(ValidIO(new ResponseRequest(false)))
      val alloc_s6 = Flipped(ValidIO(new ResponseRequest(true)))
    }
    val snRxdDat = Flipped(ValidIO(new RespWithData())) // ← 下游响应
    val bypassData = Flipped(Vec(beatSize, ...)) // ← MemUnit 旁路数据
    val rnRxdDat = Flipped(ValidIO(new RespWithData())) // ← 上游响应 (Snoop)
    val txrsp = DecoupledIO(new CHIRSP()) // → 发回 Client
    val txdat = DecoupledIO(new TaskWithData()) // → 发回数据
    val urgentRead = Output(...) // → MemUnit 紧急读
  })
}

// ResponseUnit.scala:27-34 — 响应状态机
class ResponseState(implicit p: Parameters) extends LLCBundle {
  val s_comp = Bool() // 完成阶段
  val s_urgentRead = Bool() // 紧急读
  val w_datRsp = Bool() // 等待数据响应
  val w_snpRsp = Bool() // 等待 Snoop 响应
  val w_compack = Bool() // 等待 CompAck
  val w_comp = Bool() // 等待 Comp
}
```

OpenNCB数据流



OpenNCBImp

OpenNCB 的外层封装，配置 NCB200 参数，并连接 CHI 和 AXI 接口

实例化 NCB200

```
// src/main/scala/openLLC/utils/OpenNCB.scala:67
val uNCB200 = Module(new NCB200()(new Config((_, _, _) => {
  ...
})))
```

这里将 XSCache 的 AXI、CHI 和 OpenNCB 配置传给 NCB200。

连接 CHI

```
// src/main/scala/openLLC/utils/OpenNCB.scala:98-103
Seq(
  (chi2ncb.rxreq, chi.tx.req),
  (chi2ncb.rxdnt, chi.tx.dnt),
  (chi2ncb.txrsp, chi.rx.rsp),
  (chi2ncb.txdat, chi.rx.dat)
).foreach { case (e, t) =>
  e.flitpend <> t.flitpend
  e.flitv <> t.flitv
  e.flit <> t.flit
  e.lcrdv <> t.lcrdv
}
```

对应关系

CHI请求 → NCB200
CHI写数据 → NCB200

CHI响应 ← NCB200
CHI读数据 ← NCB200

NCB200

OpenNCB 的核心模块，负责将 CHI NoSnP 请求转换为 AXI 读写事务，并将 AXI 返回转换为 CHI 响应

定义 CHI 和 AXI 接口

```
// OpenNCB/src/main/scala/openncb/NCB200.scala:18-29
class NCB200(implicit val p: Parameters)
  extends Module
  with WithAXI4Parameters
  with WithCHIParameters
  with WithNCBParameters {

  val io = IO(new Bundle {
    val chi = CHISNFInterface()
    val axi = AXI4InterfaceMaster()
  })
}
```

连接 CHI 通道

```
// OpenNCB/src/main/scala/openncb/NCB200.scala:95-98
io.chi.rxreq <> uRXREQ.io.rxreq
io.chi.rxdat <> uRXDAT.io.rxdat
io.chi.txrsp <> uTXRSP.io.txrsp
io.chi.txdat <> uTXDAT.io.txdat
```

对应关系

rxreq: 接收 CHI 读写请求
rxdat: 接收 CHI 写数据

txrsp: 返回 CHI 完成响应
txdat: 返回 CHI 读数据

连接 AXI 通道

```
// OpenNCB/src/main/scala/openncb/NCB200.scala:121-125
io.axi.aw <> uAW.io.aw
io.axi.w <> uW.io.w
io.axi.b <> uB.io.b
io.axi.ar <> uAR.io.ar
io.axi.r <> uR.io.r
```

对应关系

CHI ReadNoSnP
→ NCB200
→ AXI AR

AXI R
→ NCB200
→ CHI CompData

CHI WriteNoSnP + CHI WriteData
→ NCB200
→ AXI AW + AXI W

AXI B
→ NCB200
→ CHI 完成响应

AXI4MasterNode

OpenNCB 的 AXI Master 输出接口，负责将 AXI 请求发送给内存，并接收内存返回的数据和响应

定义 AXI4MasterNode

```
// src/main/scala/openLLC/utils/OpenNCB.scala:34-41
val axi4node = AXI4MasterNode(Seq(AXI4MasterPortParameters(
  Seq(AXI4MasterParameters(
    name = "LLC",
    id = IdRange(0, ncbParams.outstandingDepth),
    aligned = true,
    maxFlight = Some(ncbParams.outstandingDepth)
  ))
)))
```

其中：

id	→ AXI事务编号范围
aligned	→ 请求地址对齐
maxFlight	→ 最多同时处理的事务数量

对应关系

AXI	AR	→	发送读地址
AXI	R	←	接收读数据
AXI	AW	→	发送写地址
AXI	W	→	发送写数据
AXI	B	←	接收写响应

OpenLLC 将 CHI NoSnp 请求发送给 OpenNCBImp，NCB200 将请求转换为 AXI 读写事务，再通过 AXI4MasterNode 访问内存；内存响应沿相反方向转换回 CHI 响应

XSCache 整体总结



各模块作用：

CoupledL2：

处理L1请求，完成L2命中访问；
未命中时向OpenLLC发送CHI请求。

OpenLLC：

作为共享末级Cache，缓存多个L2访问的数据；
未命中时继续向内存发送请求。

OpenNCB：

将OpenLLC的CHI NoSnp请求转换为AXI事务，
通过AXI接口访问Memory。

XSCache 的核心不是单个 Cache 模块，而是由多级缓存、并发事务管理、一致性协议和内存接口共同构成的完整数据访问系统